

Software Feature Detection using Source Code Minimaps as Visual Signatures

Munif Gebara Jr.¹, Igor S. Wiese^{*3}, and Yandre M. G. Costa^{+1,2}.

¹ Graduate Program in Computer Science, State University of Maringá, Maringá, Brazil.

² Department of Informatics, State University of Maringá, Maringá, Brazil.

³ Federal University of Technology (UTFPR), Campo Mourão, Brazil.

✉munifgebara@gmail.com, *igor.wiese@gmail.com, +yandre@din.uem.br

Abstract—This study introduces a novel approach for software feature detection using source code minimaps as visual signatures. Although software analysis tools exist, to our knowledge, no work leverages visual abstractions such as minimaps combined with texture analysis for source code stereotype detection. These minimaps provide a compact visual abstraction of source code, encoding its structural and textual patterns, which are then analyzed using texture-based techniques. The proposed method employs Local Binary Patterns (LBP) to extract texture features from minimaps. It utilizes machine learning models, including K-Nearest Neighbors (KNN), Random Forest (RF), and Support Vector Machines (SVM), to classify software components into predefined stereotypes. A custom dataset was created from real-world software projects, including fixed-size and variable-size minimap representations, enabling a comparative evaluation of the method. Experimental results demonstrate the approach's efficacy, with classification accuracies reaching up to 93% for RF. The findings highlight the potential of source code minimaps combined with machine learning (ML) as a scalable solution for software analysis, opening pathways for further exploration in feature detection and classification in diverse programming environments.

Index Terms—Source code analysis, feature detection, source code minimaps, static code analysis, Local Binary Patterns, and software visualization.

I. INTRODUCTION

In software engineering, stereotypes represent recurring characteristics, behaviors, or roles associated with specific software components or systems, often used to convey higher-level abstractions. These stereotypes bridge conceptual design and implementation, helping developers and architects understand and communicate the purpose or functionality of various code elements. Closely related to design patterns, stereotypes provide additional context by categorizing software components within broader architectural frameworks [1]. For instance, recognizing that a class exhibits the behavior of a “singleton” or “observer” aids in aligning implementation with well-established design principles. This classification fosters code reusability, maintainability, and consistency, enabling more efficient collaboration and decision-making throughout the software development lifecycle. Identifying stereotypes is thus critical for optimizing design processes and enhancing the overall quality of software systems [2].

Having a mechanism to identify potential deviations from established stereotypes is crucial for maintaining software quality and implementation standardization, particularly in collaborative projects involving numerous developers. Such a mechanism ensures adherence to expected design patterns and architectural principles. This reduces the risk of inconsistencies or anti-patterns compromising maintainability and scalability. In large development teams, where varying levels of expertise and individual coding styles may lead to discrepancies, automated identification of stereotype deviations acts as a safeguard. It promotes adherence to coding standards, facilitates code reviews, and supports early detection of potential design flaws.

By preserving the alignment with established software stereotypes, this approach enhances team cohesion, streamlines integration processes, and ultimately contributes to the delivery of robust and reliable software systems.

Automated source code analysis methods, such as those employed by tools like SonarQube, systematically evaluate software artifacts to identify potential issues related to code quality, maintainability, and adherence to best practices. These tools operate by parsing source code and applying predefined rules or metrics to detect vulnerabilities, code smells, and deviations from standard design patterns. For example, SonarQube analyzes code against a set of built-in or customizable quality profiles, flagging violations such as missing comments, overly complex methods, or inefficient algorithms. Importantly, tools like SonarQube are extensible, allowing new patterns to be integrated into their rule sets.

Developers or organizations can define custom quality gates tailored to specific project needs, incorporating new stereotypes or patterns as they emerge. To effectively define new patterns within these tools, it is essential to have a deep understanding of the programming language and the associated design patterns used in the project. This knowledge accurately translates conceptual stereotypes and coding principles into specific, actionable rules that the analysis tool can apply. Without this expertise, there is a risk of misinterpreting the intent of a design pattern or incorrectly implementing a rule, which could lead to false positives or negatives in the analysis. Therefore, expertise in language and design patterns ensures that custom rules align with the intended architecture and support the overarching goals of maintaining code quality, consistency, and adherence to best practices.

This article proposes a method for source code analysis capable of automatically learning to identify stereotypes from software projects, eliminating the need for manual intervention by specialists to create or update analysis rules. By leveraging ML techniques, the proposed approach adapts to software systems' inherent variability and evolution, identifying stereotypes as patterns of features present in the source code. This automation significantly reduces the dependency on domain experts for defining or maintaining rule sets, addressing one of the primary limitations of current tools. Moreover, the ability to dynamically learn and generalize stereotypes from real-world projects ensures that the method remains relevant and effective, even as coding practices and design paradigms evolve. Thus, this work aims to enhance the flexibility and scalability of source code analysis while maintaining high standards of quality and consistency in software development.

The method uses a source code mini-map approach to feature detection. Source code mini-maps are visual abstractions of the codebase that encapsulate essential static characteristics, enabling

straightforward identification of features. By focusing on static code elements such as programming languages used, code stereotypes (e.g., classes, interfaces, methods), and other code metrics, the proposed approach tries to provide a scalable and efficient solution to feature detection.

II. RELATED WORK

Software visualization has long been a critical tool in supporting program comprehension, particularly in feature evolution and maintenance, as demonstrated by Novais et al. [3] with the SourceMiner Evolution (SME) tool. SME provides multiple interactive and coordinated views to analyze the evolution of feature code, allowing developers to track changes across software versions efficiently. By leveraging techniques such as Treemaps, Polymetric views, and dependency graphs, SME highlights the power of visualization in uncovering the structure and evolution of features within complex systems. Building on this foundation, our work extends the application of visualization by incorporating ML into source code minimap analysis. While tools like SME rely on explicit mappings and predefined views, our method automates the identification of software stereotypes through texture-based analysis, eliminating manual configuration. This integration of visualization and automated learning aims to provide scalable insights into software structure and evolution, enhancing the ability to maintain and understand complex software systems over time.

Aniche et al. [2] present a catalog of code smells tailored specifically for Model-View-Controller (MVC) architectures, demonstrating the significance of architectural-specific practices in detecting code anomalies. Their work underscores how unique architectural roles, such as controllers and repositories, exhibit specialized patterns of poor design that traditional code smell catalogs may overlook. Similarly, our proposed method emphasizes the identification of software stereotypes but diverges by leveraging automated learning techniques. Instead of relying on predefined catalogs derived from expert surveys and interviews, our approach utilizes source code minimaps and ML to learn and detect stereotypes dynamically directly from project data. This eliminates the need for manual specification of patterns, offering a scalable solution that adapts to evolving software practices, including architectural-specific challenges highlighted by Aniche et al.

The SARF Map technique, as proposed by Kobayashi et al. [4], exemplifies how visualization can enhance the understanding of software architecture by extracting and presenting implicit features and dependencies through metaphors like city maps. Their use of clustering algorithms to group related classes into visual blocks provides valuable insights for high-level decision-making and stakeholder communication. Similarly, our work builds on leveraging visualization for software analysis but extends it by introducing automated learning mechanisms to identify software stereotypes directly from source code minimaps. Unlike SARF Map, which relies on predefined algorithms and domain-specific settings, our method employs ML to dynamically recognize and classify stereotypes, providing a scalable and adaptable solution for evolving software projects. This approach addresses the challenges of manual configuration and facilitates broader applicability across diverse software systems, complementing the goals of feature abstraction and architectural clarity demonstrated by SARF Map.

Costa et al. [5] explore transforming non-visual data, specifically audio signals, into visual representations using spectrograms for music genre classification. This approach highlights the potential of translating complex, non-visual phenomena into a visual domain

that can be analyzed using texture-based methods and ML. Similarly, our work transforms source code into visual representations, known as source code minimaps, to capture its structural and textual properties. Like spectrograms, which encode time-frequency characteristics, source code minimaps provide a spatial and texture-based representation of programming constructs, enabling the application of visual pattern recognition techniques. This shared methodology demonstrates the power of visualization in bridging domains and extracting meaningful features from otherwise abstract datasets.

Costa et al. [6] used LBP for texture feature extraction from spectrograms, achieving significant results on music genre classification. By combining LBP with shallow ML techniques such as SVM, the authors highlight the effectiveness of texture descriptors in capturing discriminative patterns in visual data. In our work, we leverage LBP to extract textural features from source code minimaps, enabling the automated learning of software stereotypes. This parallel use of LBP across different domains underscores its versatility and demonstrates its potential when coupled with advanced ML models to address complex classification tasks.

III. METHODOLOGY

The proposed method consists of three main phases: (A) generating visual representations of source code using minimaps, (B) extracting textural features using LBP, and (C) classifying these features using ML models. The method aims to automate the detection process, reducing the dependency on expert-defined rules and adapting dynamically to various software projects.

The *os* library was used to manage file paths and traverse directories for loading and organizing the dataset of source code minimaps. *NumPy* was employed for numerical computations and array manipulations, which were critical for handling feature vectors and other data structures. Image processing tasks were performed using *OpenCV* (cv2), including reading, resizing, and preparing minimaps for analysis. The *scikit-image* package provided the implementation of LBP, which was used to extract texture features from the minimaps. For ML tasks, the *scikit-learn* library played a central role: *Grid-SearchCV* was used for hyperparameter optimization, and classifiers like SVM, RF, and KNN were utilized for the classification phase. Additionally, *Matplotlib* and *Seaborn* were employed to generate visualizations, such as confusion matrices and other performance metrics, aiding in the analysis and interpretation of results. This comprehensive suite of libraries ensured an efficient, modular, and reproducible implementation of the methodology.

A. Source Code Mini-Map Dataset Generation - Step A

As no publicly available datasets were specifically designed for training and evaluating the proposed approach, creating a custom dataset using real-world software projects was necessary. This dataset was constructed by collecting source code from a diverse set of open-source repositories, ensuring a variety of programming languages, architectural styles, and levels of complexity. Each project was processed to generate source code minimaps, which visually represent the structure and organization of the code. These minimaps were then labeled based on predefined software stereotypes, forming the basis for training and validating the ML models. This custom dataset addresses the lack of existing resources and ensures the proposed methodology is tested on realistic and representative scenarios.

Two large, private software projects were used to create the datasets for this study, both provided under the condition of maintaining complete anonymity. The first project originates from a private company specializing in enterprise resource planning (ERP) solutions

for the industrial sector. The second project is a government initiative developed by the Federal Government of Brazil. These projects, selected for their size and complexity, provided a diverse and realistic basis for generating the minimap datasets. Their inclusion ensures the methodology is tested on real-world scenarios while adhering to strict confidentiality requirements.

The dataset generation process begins with an algorithm that recursively scans the project's directory structure, searching for source code files of predetermined types; each file is transformed into a visual representation (mini-map), where the characters in the code are mapped to pixel intensities, as shown in Figure 1. This transformation retains the structural and syntactic information of the code, making it easier to extract features for classification.



Fig. 1. Minimaps generation.

The algorithm used for this process categorizes files based on their type (e.g., '.java', '.html', '.js'), filenames (e.g., Product-Controller.java, ProductTest.java) and annotations (e.g., '@Entity', '@Repository'), before generating images for each file in different directories. An anonymization step is performed to prevent reverse engineering by hashing file names and encrypting ASCII characters and in the content of the source code file, letters and numbers are translated into a single letter or a single number, while symbols are preserved. This anonymization process enables individuals and organizations to create the model without compromising the confidentiality of their source code. The methodology ensures that proprietary or sensitive information remains secure by transforming the code into visual representations (minimaps) and discarding semantic details. Furthermore, the experimental results demonstrated that the training performance with anonymized data was equivalent to that obtained with non-anonymized data, validating the effectiveness of the anonymization approach without sacrificing the model's accuracy or robustness. This ensures both privacy and collaboration in developing the proposed method.

Figure 2 illustrates the transformation of the same code snippet into a minimap. In the upper section of the figure, the method is applied without anonymization, while the lower section shows the result after applying anonymization. The experimental results demonstrate that the anonymization process does not significantly impact the model's performance. For instance, without anonymization, the average precision, recall, and F1-score achieved were 0.83, 0.83, and 0.83, respectively. With anonymization applied, the corresponding metrics slightly decreased to 0.82, 0.82, and 0.82. These values were obtained using an SVM classifier trained on a dataset containing 10 classes containing 500 examples. This experiment used only 500 examples per class, considering only classes with at least 500 examples. These marginal differences validate the anonymization strategy's effectiveness in preserving the minimaps' structural and textural characteristics, ensuring that sensitive information remains secure without compromising the accuracy or reliability of the classification.

The dataset was constructed by categorizing source code files into predefined stereotypes, resulting in 42,820 minimap images. The file types were classified such as *css*, *javabuilder*, *javaconfiguration*,

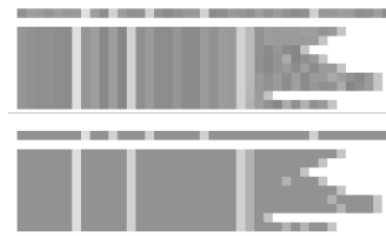


Fig. 2. Minimaps anonymization.

javacontroller, *javaconverter*, *javadataprovider*, *javado*, *javaentity*, *javaimplementation*, *javaintegrationtest*, *javainterface*, *javajasper*, *javajsf*, *javarepository*, *javarestcontroller*, *javaservice*, *javato*, *javauinitest*, *js*, *json*, *properties*, *sh*, *sql*, *svg*, *xml*, *yml*, *html*, and *javajsp*. Two separate datasets were generated: one with fixed-size minimap images of 128×128 pixels and another with variable-size images, where the width was determined by the number of columns in the longest line of the source code file and the height by the total number of lines in the file. Additionally, an 8-pixel dead zone was added to the edges of all images in both datasets to enable the usage of the LBP algorithm during texture feature extraction not discarding information close to the borders. This dual-dataset design ensures comprehensive coverage and adaptability for different machine-learning models.

B. Feature Extraction Using LBP - Step B

LBP is a robust texture descriptor extensively used in pattern recognition because it can effectively capture local texture features [7]. It has already been successfully applied in different application domains [8], [9]. LBP operates by comparing the intensity of a central pixel with its surrounding neighbors within a defined window, generating a binary code that encodes the relationship between the pixel and its neighbors. These binary patterns are then aggregated into histograms, providing a compact and discriminative representation of the texture in an image. In the context of source code minimaps, LBP serves as a critical tool for feature extraction, enabling the analysis of structural and syntactic patterns encoded in the minimaps. Costa et al. [6] demonstrated the effectiveness of LBP in capturing discriminative patterns for music genre classification from spectrograms, which are visual representations of audio signals. Similarly, in this study, LBP is applied to minimaps to extract meaningful features that reflect the visual characteristics of source code, illustrating the versatility of LBP in transforming non-visual data into a form suitable for ML.

LBP was applied to the minimap images to extract texture features, resulting in histograms that characterize the visual patterns of the source code. The average histograms for six selected classes are presented in figure 3. An example of a minimap from the respective class is displayed in the top-left corner of each histogram. These histograms highlight the distinct textural characteristics of each class, emphasizing the variability captured by the LBP method. Notably, the histogram for the JavaEntity class shows a clear difference from the histogram for the SVG class, reflecting their contrasting structural patterns. However, the JavaEntity histogram closely resembles the histogram for the JavaImplementation class, indicating a degree of similarity in their underlying texture distributions. These observations validate the ability of LBP to capture and distinguish between the unique visual features of different source code classes.

C. Classification Models - Step C

We evaluated three ML models for the classification of source code mini-maps: KNN: A simple instance-based learning method where the classification is based on the majority class among the nearest

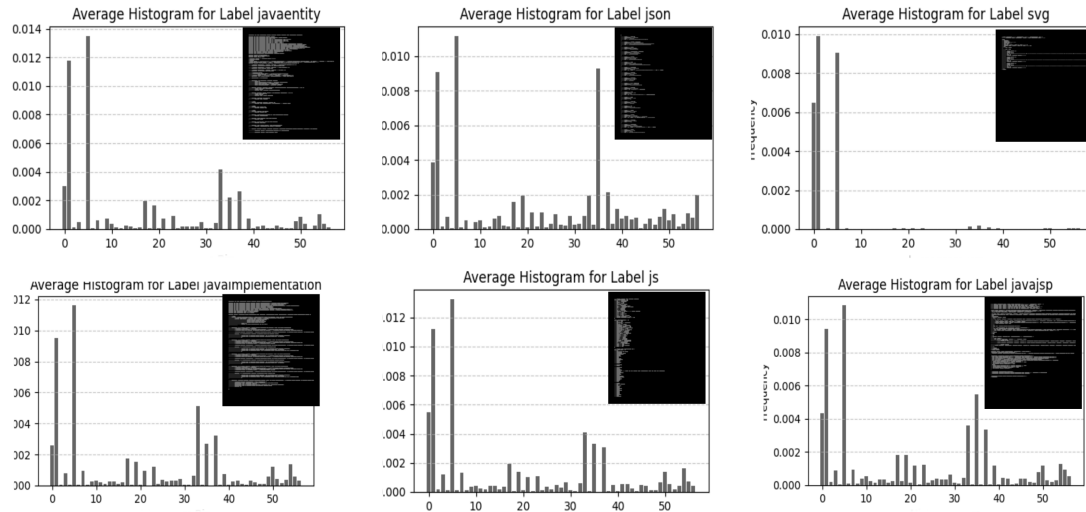


Fig. 3. Average histograms.

neighbors; RF: An ensemble learning method that constructs multiple decision trees and outputs the majority vote of the trees; SVM: A classifier that separates the data points into classes using hyperplanes in a high-dimensional space. We optimized the SVM with a radial basis function (RBF) kernel.

The KNN algorithm was employed in this study to classify the extracted texture features. A grid search was performed using GridSearchCV to optimize its performance to identify the best combination of hyperparameters. The parameter grid used for tuning included the number of neighbors with values ranging from 3 to 19 in increments of 2, and the weighting scheme (weights), which could be either uniform or distance. The grid search systematically evaluated these combinations to find the configuration that maximized the classification accuracy on the validation dataset. This approach ensures that the KNN model is appropriately tuned to capture the relationships in the data while maintaining robustness and generalization.

Similarly, the RF algorithm was applied for classification, with hyperparameter tuning conducted using GridSearchCV to maximize its effectiveness. The parameter grid for this model included many estimators representing the number of trees in the ensemble, with values of 10, 50, 100, 200, 500, and 1000, and max depth, which determines the maximum depth of each tree, explored with values of None (unrestricted depth) and fixed depths of 10, 20, 30, and 100. A systematic evaluation of these parameter combinations selected the best configuration based on validation accuracy. This ensured the RF classifier was robust and well-optimized, balancing model complexity and generalization.

Building on the use of hyperparameter tuning, the SVM algorithm was also employed for classification, with GridSearchCV utilized to optimize its parameters. The parameter grid for SVM included the regularization parameter C , tested with values of 1000, 2000, and 10000, which controls the trade-off between achieving a low error on the training data and minimizing model complexity. Additionally, the gamma parameter, which defines the influence of a single training example, was explored with values of 2, 1, 0.1, 0.01, 0.001, and 0.0001. The kernel function was set to the radial basis function (RBF), particularly suited for capturing complex decision boundaries. The grid search systematically evaluated all combinations of these parameters to identify the configuration that provided the highest accuracy on the validation set. It ensured the SVM model was finely tuned to the dataset's characteristics while maintaining its

generalization capacity.

IV. EXPERIMENTAL RESULTS

To compare the training algorithms and evaluate the impact of fixed-size versus variable-size minimaps, we selected only classes with more than 1000 examples to ensure sufficient representation. Each class was limited to 1000 randomly selected examples from the available software project instances, maintaining dataset balance. This process resulted in a dataset with seven representative classes: file types JSON, SVG, JavaScript (JS), JSP, and three Java stereotypes – Entity, UnitTests, and ServicesImplementations. This selection ensured a diverse yet balanced dataset, capturing meaningful patterns while minimizing biases from uneven class distributions.

For the experimental evaluation, two datasets were created with identical class distributions and sample sizes. The key difference was in the minimap image sizes. In the first dataset, images had variable dimensions based on the structure of the source code files, with width corresponding to the longest line and height to the total number of lines. In contrast, the second dataset contained fixed-size 128×128 pixel images, often capturing only the source code's initial portion. The fixed-size dataset was designed to enable comparisons with methods that process raw images, such as artificial neural networks, rather than relying on texture-based features like LBP histograms. This distinction ensures adaptability across different machine-learning approaches. (Figure 4)

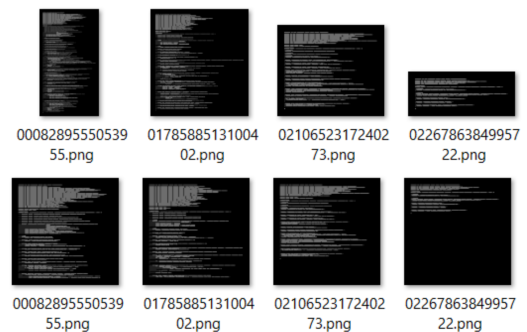


Fig. 4. Variable versus Fixed minimaps sizes.

The KNN algorithm results showed a noticeable difference between the variable-size and fixed-size datasets. For the variable-size dataset, KNN achieved an overall accuracy of 79%, in contrast,

with the fixed-size dataset, KNN's performance slightly decreased, achieving an accuracy of 78%.

RF consistently outperformed KNN on both datasets. With the variable-size dataset, RF achieved an accuracy of 93%. The performance of the fixed-size dataset was nearly identical, with an overall accuracy of 93%.

The SVM classifier also demonstrated strong performance, particularly with the variable-size dataset, where it achieved an accuracy of 89%. For the fixed-size dataset, SVM achieved a slightly lower accuracy of 88%. The reduced size of the minimap images appeared to negatively impact the ability to distinguish between classes such as 'javaunittest' and 'javajsp'.

Overall, the results across all algorithms and datasets indicate a consistent trend: variable-size datasets offer slightly better performance metrics than fixed-size datasets. This can likely be attributed to the richer information retained in the variable-size minimaps. Nonetheless, the fixed-size dataset's performance remains competitive and holds promise for applications involving image-based neural network classifiers, as it simplifies preprocessing and enables compatibility with standardized input dimensions.

V. DISCUSSION

The results of this study demonstrate the potential of source code minimaps as an approach for software feature detection. By leveraging visual abstractions of source code and analyzing their texture properties with Local LBP, the methodology bridges the gap between visual representation and ML-based classification. The comparative analysis of fixed-size and variable-size datasets highlights important trade-offs. While fixed-size minimaps offer standardization and compatibility with image-based neural networks, variable-size minimaps retain richer contextual information that enhances classification accuracy. These findings suggest that the choice between dataset types depends on the specific application and the desired balance between accuracy and scalability.

Moreover, the high classification accuracy of RF across both datasets underscores its robustness and suitability for this task. However, the slightly lower performance of KNN and SVM reveals potential limitations of traditional ML methods when dealing with high-dimensional texture features. The results also raise interesting questions about the applicability of deep learning methods, which may offer improved performance by learning hierarchical representations directly from minimaps without relying on predefined descriptors like LBP. This opens the door to further exploration of advanced architectures, such as convolutional neural networks (CNNs), for direct minimap analysis.

VI. CONCLUSION

This paper presents a novel approach based on texture properties for software feature detection using source code minimaps and ML classifiers. The results demonstrate that the proposed methodology can accurately classify software components into stereotypes, achieving up to 93% accuracy using RF. The comparison between fixed-size and variable-size minimaps revealed that they slightly outperform their fixed-size counterparts due to preserving richer structural details from the source code. Nonetheless, the fixed-size minimaps remain viable for applications requiring standardization and preprocessing for deep learning models.

In summary, the findings validate the feasibility and effectiveness of the proposed method for automated software feature detection. The ability to transform non-visual data, such as source code, into visual signatures and classify them with high accuracy offers a new perspective for static code analysis and software engineering research.

VII. FUTURE WORK

As this study represents the first exploration of source code classification using minimaps, numerous questions remain open for future research. One avenue involves testing other texture descriptors beyond LBP and exploring its variations to assess their impact on classification performance. Another critical step is the creation of datasets enriched with additional metadata, such as code smells and adherence to best practices, which could provide a richer context for classification tasks. Future efforts should also focus on building more diverse datasets, including general-purpose datasets encompassing multiple programming languages or specialized datasets tailored to a single programming language to evaluate performance in more controlled scenarios. Moreover, experiments with deep learning models that can directly process minimap images should be conducted.

Other promising directions for future research include investigating the scalability of the proposed methodology when applied to significantly larger datasets to evaluate its robustness in real-world scenarios. Another important aspect is integrating dynamic analysis information, such as runtime behavior, into the classification process to complement the static features extracted from the minimaps. Furthermore, developing techniques to identify the minimaps' most discriminative regions automatically could enhance the classifiers' effectiveness while reducing computational costs. Finally, extending this methodology to other domains, such as detecting vulnerabilities or identifying architectural patterns, could broaden its applicability and further validate its utility in software engineering tasks.

Lastly, the source code and datasets used in this study are publicly available at <https://anonymous.4open.science/r/codeminimap-5FB2/README.md> this repository.

ACKNOWLEDGMENT

Study partially financed by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) - Finance Code 001.

REFERENCES

- [1] X. Guan and C. Treude, "Enhancing source code representations for deep learning with static analysis," in *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension (ICPC '24)*. IEEE/ACM, 2024. [Online]. Available: <https://dl.acm.org/doi/abs/10.1145/3643916.3644396>
- [2] M. Aniche, G. Bavota, C. Treude, M. A. Gerosa, and A. van Deursen, "Code smells for model-view-controller architectures," *Empirical Software Engineering*, vol. 23, no. 4, pp. 2121–2157, 2018.
- [3] R. L. Novais, C. Nunes, A. Garcia, and M. Mendonça, "Sourceminer evolution: A tool for supporting feature evolution comprehension," in *2013 IEEE International Conference on Software Maintenance (ICSM)*. IEEE, 2013, pp. 508–517.
- [4] K. Kobayashi, M. Kamimura, K. Yano, K. Kato, and A. Matsuo, "Sarf map: Visualizing software architecture from feature and layer viewpoints," in *2013 IEEE International Conference on Program Comprehension (ICPC)*. IEEE, 2013, pp. 43–52.
- [5] Y. M. Costa, L. S. Oliveira, A. L. Koerich, and F. Gouyon, "Music genre recognition using spectrograms," in *2011 18th International conference on systems, signals and image processing*. IEEE, 2011, pp. 1–4.
- [6] Y. M. Costa, L. Oliveira, A. L. Koerich, F. Gouyon, and J. G. Martins, "Music genre classification using lbp textural features," *Signal Processing*, vol. 92, no. 11, pp. 2723–2737, 2012.
- [7] T. Mäenpää and M. Pietikäinen, "Texture analysis with local binary patterns," in *Handbook of pattern recognition and computer vision*. World Scientific, 2005, pp. 197–216.
- [8] L. Nanni, R. L. Aguiar, Y. M. Costa, S. Brahmam, C. N. Silla Jr, R. L. Brattin, and Z. Zhao, "Bird and whale species identification using sound images," *IET Computer Vision*, vol. 12, no. 2, pp. 178–184, 2018.
- [9] R. M. Pereira, Y. M. Costa, R. L. Aguiar, A. S. Britto, L. E. Oliveira, and C. N. Silla, "Representation learning vs. handcrafted features for music genre classification," in *2019 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2019, pp. 1–8.